

Пояснительная записка

Полуфинал Volga IT 2026, дисциплина «Искусственный интеллект и анализ данных» Задача: распознавание нестандартных ГРЗ (типы 1, 1А, 1Б по ГОСТ Р 50577-2018)

1. Постановка и итог

Решение принимает каталог изображений и формирует CSV `image;plate_num;plate_type;confidence`. Целевые классы — **type1**, **type1a** (квадратные двухстрочные), **type1b** (жёлтые), **other** (всё остальное: прицепы, мотоциклы, полиция, военные, дипломатические, транзитные, иностранные). Работает офлайн, все веса включены. Ключевая идея: дефицит редких типов закрывается данными — контролируемым генератором синтетики с заменой реальных знаков «на месте» и собранным вручную-проверенным набором реальных фото со свободными лицензиями.

Результат на отложенной реальной выборке (282 изображения, 314 целевых знаков, никогда не участвовали в обучении): полнота обнаружения 0.987, сквозная точность — знак найден И номер прочитан полностью верно — 0.943, точность определения типа 0.981. Скорость: 78 мс на изображение (медиана, GPU) и 92 мс на одном только CPU, то есть требование «не более 100 мс» выполняется даже без видеокарты.

2. Источники данных и методика сбора

Реальные данные. Единственный законный масштабируемый источник без собственной съёмки — фотобанки со свободными лицензиями. Использован Wikimedia Commons (API commons.wikimedia.org, скрипт [tools/harvest_commons.py](#)): категории и полнотекстовые запросы («Buses in Russia by city», «Taxis in Moscow», «Yandex.Taxi in Moscow», серии «Moscow, Toyota Probox ... 2025», «Vladivostok ... 2024», «License plates of Kazakhstan/Belarus/Ukraine», «Trailer/Military/Diplomatic license plates of Russia» и др.), а также API Openverse (Flickr CC). Для каждого файла сохраняются страница-источник, автор, лицензия и её URL; допускались только CC0 / Public Domain / CC BY / CC BY-SA (NC/ND, GFDL-only и несвободные отсеиваются автоматически). Фотографии с людьми как смысловым центром исключены, лица размыты детектором YUNet ([tools/blur_faces.py](#)) при сборке датасета.

Разметка. Полуавтоматическая, честно описанная: собственная модель (обученная сначала только на синтетике) предлагает четырёхугольник, тип и чтение; каждый кандидат просмотрен человеком на контактных листах ([tools/review_sheets.py](#)), текст и тип исправлены вручную, лишние кандидаты отклонены, изображение включается в датасет только если все знаки на нём проверены (иначе остались бы неразмеченные знаки). Нечитаемые позиции помечены #. Углы уточняются по выровненному кропу.

Синтетика ([generator/](#)). Процедурный рендер по геометрии ГОСТ (мм -> пиксели) шрифтом GOST-R-50577-93 (+3 запасных шрифта для устойчивости к неточному начертанию), 12 подтипов знаков. Реалистичность обеспечивают: джиттер геометрии, рамки и болты, грязь/царапины/окклюдеры (закрытые символы получают метку #), градиентное освещение и блики, случайная 3-D поза (yaw $\pm 50^\circ$, pitch $\pm 30^\circ$, roll $\pm 15^\circ$) через pinhole-проекцию, подгонка яркости к фону, ночь/дождь/снег/смаз/расфокус/шум/потеря разрешения/JPEG. Фоны — реальные фото датасета, на которых настоящие знаки заменяются синтетическими по их четырёхугольникам (так квадратный или жёлтый знак появляется на реальном автомобиле в реальной сцене), плюс процедурные фоны с текстовым «мусором» (вывески) для снижения ложных срабатываний. Набор воспроизводится командой с фиксированным `--seed`.

Важная деталь: по ГОСТ тип 1Б имеет формат **ББ ЦЦЦ РР**, а задание описывает для 1Б маску **Б ЦЦЦ ББ РР**. В синтетике генерируются оба формата, грамматика решения принимает оба.

3. Архитектура и обоснование

Пайплайн из трёх лёгких моделей (все — PyTorch, суммарно < 10 МБ):

1. Детектор YOLO11n-pose (2.9 М параметров), дообученный на 4 класса с 4 ключевыми точками = углами знака. Один проход даёт bbox, тип знака и точный четырёхугольник. Почему pose, а не обычный детектор + отдельная сеть углов: экономия ~5 мс и точное выравнивание перспективы — главный фактор качества OCR при съёмке под углом. Класс детектора используется как априорная оценка типа; окончательный тип принимается модулем [lpr/typing.py](#) по совокупности признаков: геометрия четырёхугольника (двухстрочный => 1А), цвет поля (жёлтый => 1Б, синий/красный/чёрный => прочие) и соответствие прочитанной строки маскам. Так белые иностранные знаки (BY, KZ, UA), неотличимые от типа 1 по внешнему виду, отсеиваются по грамматике, а не «угадываются» детектором.
2. Выравнивание гомографией в канонический прямоугольник; для 1А — резка на две строки с перекрытием, каждая строка читается тем же распознавателем (одна модель для всех типов).
3. CRNN (CNN -> BiLSTMx2 -> CTC, 1.9 М параметров, вход 32x160, алфавит 10 цифр + 12 букв + #). Класс # обучается на реально закрытых позициях синтетики, дополнительно порог уверенности символа.
4. Грамматика и пост-обработка: две маски (задание и ГОСТ), позиционное исправление похожих символов, штраф уверенности за неправдоподобную строку, перевод в **other** при явном несоответствии.
5. Офлайн-гарантия: пакет [lpr](#) при импорте выставляет `YOLO_OFFLINE=1` и `YOLO_AUTOINSTALL=false`, поэтому Ultralytics не делает проверок версий, не шлёт аналитику и ничего не скачивает; все веса лежат в `weights/`. Внешних API решение не использует.
6. Гейт «знак на ТС»: COCO YOLO11n (car/bus/truck/motorcycle) — знак вне ТС получает понижение уверенности либо исключается (`--drop-off-vehicle`). Это закрывает требование п. 7 (рекламные щиты, витрины) без обучения.

Расширяемость. Новый тип знака = новый класс детектора + новый builder в `plate_render.py` (30–50 строк: размеры, раскладка, цвета) + маска в `plates.py`. Распознаватель и выравнивание не меняются; для двух-/трёхстрочных знаков задаётся схема резки строк. Иностранные знаки других стран и старые советские типы добавляются так же; уже сейчас генератор рендерит KZ/BY/UA/UZ/KG/AM/GE/EU как класс `other`.

3а. Состав датасета

Реальные данные — изображений: 2682, знаков: 3189

Тип	Изображений	Знаков	Уникальных номеров
type1	1885	2209	1768
type1a	86	86	62
type1b	328	343	303
other	497	551	476

Синтетические данные — изображений: 5135, знаков: 8584

Тип	Изображений	Знаков	Уникальных номеров
type1	1729	1926	1910
type1a	2343	2802	2800
type1b	2186	2605	2572
other	1159	1251	1241

Лицензии реальных изображений (по знакам): CC0 — 1620, CC BY 4.0 — 715, CC BY-SA 4.0 — 543, CC BY-SA 3.0 — 112, CC BY 2.0 — 89, CC BY 3.0 — 67, CC BY-SA 2.0 — 29, Public domain — 12, CC BY 3.0 de — 1, CC BY-SA 2.5 — 1

Источники: commons.wikimedia.org — 3153, www.flickr.com — 36

Условия съёмки (все данные): day — 9879, angle — 4882, dirt — 4882, glare — 3647, night — 1894, motion_blur — 1422, rain — 1246, snow — 1050

is_vehicle (реальные): 1 — 3171, 0 — 18

Знаков с нечитаемыми позициями (#) в реальных данных: 42

Отчёт валидатора: `dataset/validation_report.txt` (0 ошибок). Рекомендованный минимум по уникальным номерам 1A (50) выполнен (57), по числу изображений 1A (150) — нет (80): квадратные знаки в открытых фотобанках крайне редки; недостаток закрыт 2343 синтетическими сценами с квадратными знаками на реальных автомобилях.

4. Обучение

Обучение шло итеративно, по схеме «синтетика -> разметка реальных -> дообучение» (все запуски — `train/train_detector.py`, `train/train_ocr.py`, Apple M4 / MPS):

Этап	Данные	Эпохи	Назначение
Детектор v0	9 000 синтетических сцен (процедурные фоны + верхние части реальных фото)	6	предложения для ручной разметки
OCR v0	31 k синтетических кропов	12	чтение кандидатов при разметке
Детектор v1	892 реальных (x3) + 4 000 синтетических на реальных фонах + 3 000 bootstrap	12	второй круг разметки
OCR v1	60 k синт. кропов + 8 k кропов из сцен + 1.5 k реальных (x6)	25	93.6 % точных чтений на реальной валидации
Детектор v2 (финал)	датасет (2 682 реальных x2, 5 135 синтетических) + 7 000 доп. синтетических, 15 % реальных отложено	10 (дообучение v1)	итоговые веса <code>weights/det.pt</code>
OCR v2 (финал)	то же + реальные кропы датасета (x4), валидация на отложенных	15 (дообучение v1)	итоговые веса <code>weights/ocr.pt</code>

Детектор: YOLO11n-pose, 640 px, batch 32, мозаика, HSV/масштаб/сдвиг, без горизонтального отражения (зеркальные знаки не встречаются), `kpt_shape` [4,2]. Распознаватель: CTC, AdamW + OneCycle, аугментации кропов (аффинные, смаз, шум, инверсия для ИК-камер). Отложенная тестовая выборка — 15 % реальных изображений (детерминированный хэш имени), исключённых из финального дообучения; из неё дополнительно убраны изображения, которые видели модели первого круга (итого 282 изображения, 316 знаков).

4.1. Отложенная реальная выборка

282 изображения / 314 целевых знаков; эти изображения целиком исключались из обучения детектора.

Итог по целевым типам (1, 1A, 1B): полнота обнаружения 0.987, точное совпадение номера 0.955 от найденных, сквозная точность (найден И прочитан полностью верно) 0.943, посимвольная точность 0.989, точность определения типа 0.981.

Тип	Знаков в эталоне	Найдено	Полнота	Точный номер	С учётом #	Посимвольно	Ложных
type1	266	262	0.985	0.954	0.958	0.988	14
type1a	6	6	1.000	0.833	1.000	0.981	1
type1b	42	42	1.000	0.976	1.000	0.997	1
other	24	16	0.667	0.188	0.250	0.675	7
целевые	314	310	0.987	0.955	0.965	0.989	16

Знаков типа **other** (прицепы, мотоциклы, иностранные), ошибочно выданных с целевым типом: 2 из 24.

Зависимость от размера знака в кадре (целевые типы):

Ширина знака	Знаков	Полнота	Сквозная точность
<32 px	2	1.000	1.000
32–64 px	75	0.987	0.893
64–128 px	130	0.992	0.946
≥128 px	107	0.981	0.972

Зависимость от условий съёмки (целевые типы; у знака может быть несколько меток):

Условие	Знаков	Полнота	Сквозная точность
день	313	0.987	0.942
ракурс	58	0.983	0.897
блики	30	0.967	0.967
смаз	15	1.000	0.867
загрязнение	10	1.000	0.800
ночь	1	1.000	1.000

Матрица ошибок по типам (строка – эталон, столбец – решение):

эталон \ решение	type1	type1a	type1b	other	не найден
type1	258	0	2	2	4
type1a	1	5	0	0	0
type1b	1	0	41	0	0
other	2	0	0	14	8

Про «ложные срабатывания». Первый замер дал 38 непарных предсказаний на целевых типах. Проверка показала, что это в основном не ошибки решения, а пропуски разметки: стартовый детектор, которым предлагались кандидаты на ручную проверку, просто не находил часть знаков, поэтому они никогда не попадали на контактные листы. Финальным детектором был сделан контрольный проход по всем реальным изображениям датасета; 224 найденных неразмеченных кандидата просмотрены человеком, 180 подтверждены и добавлены в разметку. После этого число непарных предсказаний на целевых типах упало с 38 до 16 – приведённая выше таблица посчитана по исправленной разметке. Для полной прозрачности: по исходной разметке те же самые предсказания давали полноту 0.986, точность номера 0.951 и сквозную точность 0.938.

4.2. Синтетический стресс-тест

Реальная отложенная выборка содержит мало квадратных знаков (5 шт.), поэтому дополнительно измерено качество на независимом синтетическом наборе (seed 777, 694 изображения, 1159 знаков, из них 394 квадратных): он сгенерирован тем же генератором, но с другим seed, на фонах только из отложенных реальных изображений, и никогда не участвовал в обучении. Набор намеренно тяжелее реального: доля ночи, дождя, снега, смаза, сильных ракурсов и мелких знаков в нём в разы выше.

Итог по целевым типам (1, 1A, 1B): полнота обнаружения 0.806, точное совпадение номера 0.718 от найденных, сквозная точность (найден И прочитан полностью верно) 0.579, посимвольная точность 0.928, точность определения типа 0.963.

Тип	Знаков в эталоне	Найдено	Полнота	Точный номер	С учётом #	Посимвольно	Ложных
type1	208	164	0.788	0.756	0.848	0.939	23
type1a	394	330	0.838	0.700	0.824	0.923	9
type1b	372	291	0.782	0.718	0.814	0.929	12

Тип	Знаков в эталоне	Найдено	Полнота	Точный номер	С учётом #	Посимвольно	Ложных
other	185	117	0.632	0.231	0.256	0.612	30
целевые	974	785	0.806	0.718	0.825	0.928	44

Знаков типа **other** (прицепы, мотоциклы, иностранные), ошибочно выданных с целевым типом: 22 из 185.

Разница с реальной выборкой закономерна: синтетика намеренно набита тяжёлыми случаями (ночь, дождь, снег, смаз, ракурсы до 50°, знаки шириной от 22 px), которых в открытых фотобанках почти нет. Столбец «с учётом #» показывает 0.825 — то есть значительная часть расхождений приходится на позиции, которые генератор сам пометил как закрытые грязью или бликом.

5. Замеры скорости

Замеры **tools/benchmark.py** (среднее на изображение, включая детектор, гейт «на ТС» при 416 px и OCR; пайплайн работает целиком на одном устройстве):

Устройство	Вход детектора	Среднее, мс	Детектор	Гейт	OCR
Apple M4, GPU (MPS)	960	110.5	43.7	22.4	46.0
Apple M4, CPU (8 ядер)	960	92.1	66.8	19.6	6.4
Apple M4, CPU (8 ядер)	640	55.1	32.3	18.5	5.6

Медиана на GPU — 78 мс (среднее выше из-за первых кадров), на CPU при 960 px — 92 мс, при 640 px — 55 мс. То есть требование «не более 100 мс на изображение» выполняется даже на одном только CPU в настройках по умолчанию.

Оценка для референсной конфигурации жюри (i5-7600 + GTX 1050 Ti 4 ГБ): детектор YOLO11n при 960 px в PyTorch/CUDA занимает 25–40 мс, гейт «на ТС» при 416 px — 6–10 мс, CRNN на 1–3 кропах — 3–5 мс; ожидаемое среднее 40–60 мс/изображение. Если запас потребуется увеличить, есть три независимых рычага без переобучения: `--imgsz 640` (примерно вдвое быстрее, полнота на знаках уже 32 px по ширине ниже), `--no-vehicle-gate` (минус ~20 мс), `--half` (FP16 на CUDA) и экспорт в ONNX/TensorRT (**tools/export_onnx.py**). Размер моделей: детектор 5.6 МБ, распознаватель 8.2 МБ, COCO-детектор 5.4 МБ — суммарно 19 МБ, что свободно помещается в 4 ГБ видеопамати GTX 1050 Ti.

6. Известные ограничения

- Реальных квадратных знаков (1A) в открытых фотобанках мало; их доля в обучении в основном синтетическая (заменой на реальные автомобили). На очень мелких знаках (< 25 px по ширине) чтение ненадёжно — тогда выводятся #.
- Гейт «на ТС» опирается на COCO-детектор: знак на машине, целиком не попавшей в кадр (только бампер), может быть помечен как «не на ТС» -> только понижение уверенности, не удаление (по умолчанию).
- Жёлтые знаки формата ГОСТ (**ББ ЦЦЦ РР**) выводятся как прочитано (7–8 символов), что расходится с маской из задания; при необходимости это переключается в **plates.py**.
- Лицензии CC BY-SA у части реальных фото требуют сохранения условия ShareAlike (указано в **attribution.csv**).
- Честная оговорка о методике измерения. Эпоха финального распознавателя выбиралась по точности на кропах из отложенных изображений, поэтому сквозная точность на реальной отложенной выборке может быть на доли процента оптимистичной. Контрольная проверка: на независимом синтетическом стресс-тесте, который не использовался ни в обучении, ни в выборе эпохи, предыдущая версия распознавателя (обучавшаяся вообще без реальных данных) и финальная дают одинаковый результат — 0.580 и 0.579 сквозной точности. Детектор такой оговорки не требует: 15 % реальных изображений исключались из его обучения целиком.
- В задании сказано, что трёхзначные коды региона начинаются с 1, 2 или 7; в реальности массово встречаются 977, 799 (Москва) и 550 (Московская обл.). Решение и валидатор допускают любой трёхзначный код с ненулевой первой цифрой — иначе значительная часть настоящих московских знаков была бы отброшена.

7. Идеи развития

- Трекинг в видеопотоке (RTSP) с голосованием чтений по кадрам — резко повышает точность при 100 мс/кадр.
- Экспорт детектора в TensorRT/ONNX FP16 (на GTX 1050 Ti даёт ещё x1.5–2 по скорости).
- Активное дообучение: кандидаты с низкой уверенностью автоматически попадают на ручную проверку и в датасет (инструменты **autolabel.py/review_sheets.py** уже это реализуют).
- Знаки других стран ЕАЭС как отдельные классы с собственными грамматиками.